BeautifulSoup Cheat Sheet

Python | bs4 | parsing, searching & extracting HTML

1. INSTALL & FIRST PARSE

```
# pip install beautifulsoup4 lxml requests
import requests
from bs4 import BeautifulSoup

url = "https://quotes.toscrape.com"
r = requests.get(url, timeout=10)
r.raise_for_status()
soup = BeautifulSoup(r.text, "lxml")
print(soup.title.text)

# From a local file
with open("page.html", encoding="utf-8") as f:
    soup = BeautifulSoup(f, "lxml")
```

2. PARSERS

Parser	Notes
`"html.parser"`	Built in, decent speed, lenient
`"lxml"`	Fastest, lenient (pip install lxml)
`"xml"` / `"lxml-xml"`	The only XML parser
`"html5lib"`	Most browser-like, slowest

3. QUICK TAG ACCESS

```
soup.title              # <title> tag
soup.title.string       # text inside tag
soup.a                  # first <a> on the page
soup.a["href"]          # attribute value
soup.body.h1.get_text(strip=True)
print(soup.prettify())  # indented HTML
```

4. TAG PROPERTIES

`tag.name`	Tag name
`tag.attrs`	Dict of all attributes
`tag.text` / `get_text()`	All nested text
`tag.get_text(" ", strip=True)`	Joined with space, trimmed
`tag.string`	Text if exactly one child, else None
`tag.get("href")`	Attribute or None (safe)
`tag["class"]`	List of classes (not a string!)
`tag.has_attr("id")`	Attribute exists?
`tag.contents`	List of direct children
`tag.children`	Iterator of direct children
`tag.descendants`	All nested nodes
`tag.stripped_strings`	All text pieces, trimmed

5. NAVIGATING THE TREE

`.parent` / `.parents`	Parent / all ancestors
`.find_parent("div")`	Nearest matching ancestor
`.next_sibling`	Next node (often whitespace!)
`.find_next_sibling("p")`	Next sibling tag matching
`.find_previous_sibling()`	Previous sibling tag
`.find_next("a")`	Next match anywhere after tag
`.next_element`	Next parsed node in document

6. FIND() & FIND_ALL()

```
soup.find("div")                  # first, or None
soup.find_all("a")                # list (maybe empty)
soup.find_all(["h1", "h2", "h3"])
soup.find("div", class_="card")   # note the underscore
soup.find("div", id="main")
soup.find("a", href=True)         # has an href
soup.find_all("a", attrs={"data-id": "5"})
soup.find_all("p", limit=3)
soup.find_all("li", recursive=False)
soup.find_all(string="Next")      # by exact text
soup.find_all("a", string="Next")
soup(["script", "style"])         # shortcut find_all
```

7. ADVANCED FILTERS

```
import re
soup.find_all("a", href=re.compile(r"^/page"))
soup.find_all(string=re.compile("price", re.I))

def has_price(tag):
    return (tag.name == "span"
            and tag.has_attr("data-price"))
soup.find_all(has_price)

soup.find_all("div", class_=["a", "b"])   # a OR b
soup.find_all("div", class_="a b")        # exact "a b"
```

8. CSS SELECTORS

```
soup.select("div.card a")        # list
soup.select_one("#main h1")      # first or None
soup.select("ul > li", limit=5)
for a in soup.select("a[href]"):
    print(a["href"])
```

Selector	Meaning
`tag .cls #id`	By tag, class, id
`div.card`	div with class card
`div > p`	Direct child
`div p`	Any descendant
`h2 + p`	Immediately following sibling
`h2 ~ p`	All following siblings
`a[href]`	Has attribute
`a[href="/x"]`	Exact value
`a[href^="http"]`	Starts with
`a[href$=".pdf"]`	Ends with
`a[href*="shop"]`	Contains
`li:first-child` / `:nth-of-type(2)`	Position
`a:not(.x)`	Negation
`h1, h2`	Either selector
`p:-soup-contains("hi")`	Text contains

9. EXTRACTION PATTERNS

```
# Absolute links
from urllib.parse import urljoin
links = [urljoin(url, a["href"])
         for a in soup.select("a[href]")]

# Images
imgs = [i["src"]
        for i in soup.find_all("img", src=True)]

# Safe text helper
def txt(el):
    return el.get_text(strip=True) if el else None

# List of dicts from repeated cards
items = [{"name": txt(c.select_one("h2")),
          "price": txt(c.select_one(".price")),
          "url": c.a["href"] if c.a else None}
         for c in soup.select("div.product")]
```

10. TABLES

```
rows = []
for tr in soup.select("table tr"):
    cells = [c.get_text(strip=True)
             for c in tr.find_all(["td", "th"])]
    if cells:
        rows.append(cells)

# Or let pandas do it
import pandas as pd
from io import StringIO
dfs = pd.read_html(StringIO(str(soup)))   # all tables
```

11. MODIFYING THE TREE

```
tag.string = "New text"
tag["class"] = "new"
del tag["id"]
tag.decompose()          # remove and destroy
tag.extract()            # remove, keep the object
tag.replace_with(other)
new = soup.new_tag("a", href="/x")
new.string = "Link"
tag.append(new)          # or insert_after / insert_before

for s in soup(["script", "style"]):
    s.decompose()        # strip junk before get_text

with open("out.html", "w", encoding="utf-8") as f:
    f.write(soup.prettify())
```

12. SCRAPER TEMPLATE (PAGINATION)

```
import time, requests
from bs4 import BeautifulSoup

HEADERS = {"User-Agent": "Mozilla/5.0"}
base = "https://quotes.toscrape.com"
path, data = "/", []

while path:
    r = requests.get(base + path, headers=HEADERS,
                     timeout=10)
    r.raise_for_status()
    soup = BeautifulSoup(r.text, "lxml")
    for q in soup.select("div.quote"):
        data.append(q.select_one(".text").get_text())
    nxt = soup.select_one("li.next > a")
    path = nxt["href"] if nxt else None
    time.sleep(1)        # be polite
```

```
import pandas as pd
pd.DataFrame(items).to_csv("out.csv", index=False,
                           encoding="utf-8-sig")
```

13. SPEED: PARSE ONLY WHAT YOU NEED

```
from bs4 import SoupStrainer
only_a = SoupStrainer("a")
soup = BeautifulSoup(html, "lxml", parse_only=only_a)
```

14. WITH SELENIUM (JS-RENDERED PAGES)

```
driver.get(url)
# ...wait for content to load...
soup = BeautifulSoup(driver.page_source, "lxml")
```

15. COMMON PITFALLS

Problem	Fix
`find()` returns None, then AttributeError	Guard with `if el:` or a helper like `txt()`
`class` is a Python keyword	Use `class_="x"` or `attrs={"class": "x"}`
`tag["class"]` is a list	`" ".join(tag["class"])`
Data missing from the HTML	Page is JS-rendered: use Selenium, or find the JSON API in DevTools > Network
Garbled characters	Set `r.encoding = r.apparent_encoding` or pass `r.content`
403 / blocked	Send a User-Agent, slow down, check robots.txt

Problem	Fix
Messy whitespace	`get_text(" ", strip=True)`
Slow parsing	Use `"lxml"` and `SoupStrainer`

> BeautifulSoup only **parses** HTML - it can't fetch pages or run JavaScript. Pair it with `requests` (static pages) or Selenium / Playwright (dynamic pages).